

Arquitetura e Padrões de Software: A Estrutura e a Construção

Uma visão macro do desenvolvimento de sistemas

Apresentação: Grupo 3

Equipe: Rian (Líder), Julio, Henrique, Mary, Tomas



Fazer tijolos não é construir a casa



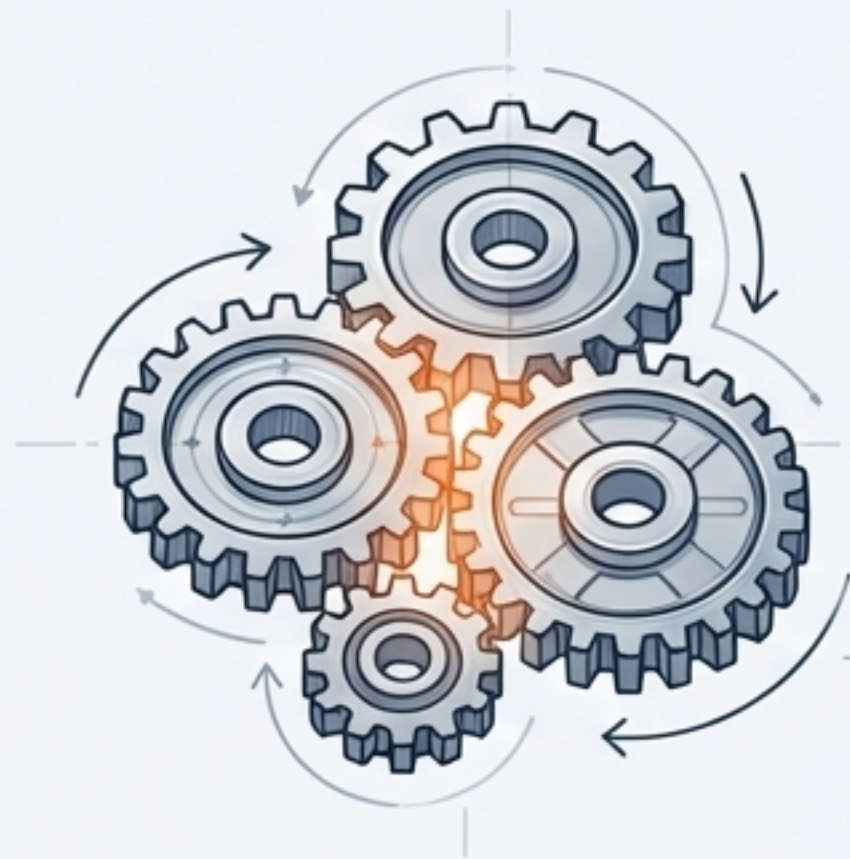
- **O Código vs. O Todo:** Conhecer regras de código (como SOLID) é essencial para fazer bons tijolos, mas não ensina a erguer o prédio.



- **A Visão Macro:** Desenvolver um software exige um Zoom Out. É preciso olhar o sistema de cima.
- **O Peso da Decisão:** Decisões arquiteturais são escolhas estruturais tecnicamente custosas e difíceis de mudar após a implementação.

O que é, afinal, a Estrutura?

A arquitetura é o nível de design que vai além dos algoritmos. Ela define o esqueleto e a integridade conceitual do produto, ocultando detalhes internos do código para organizar a interação global do sistema.



Processamento

Componentes que realizam a lógica.



Dados

Informação armazenada e transitada.

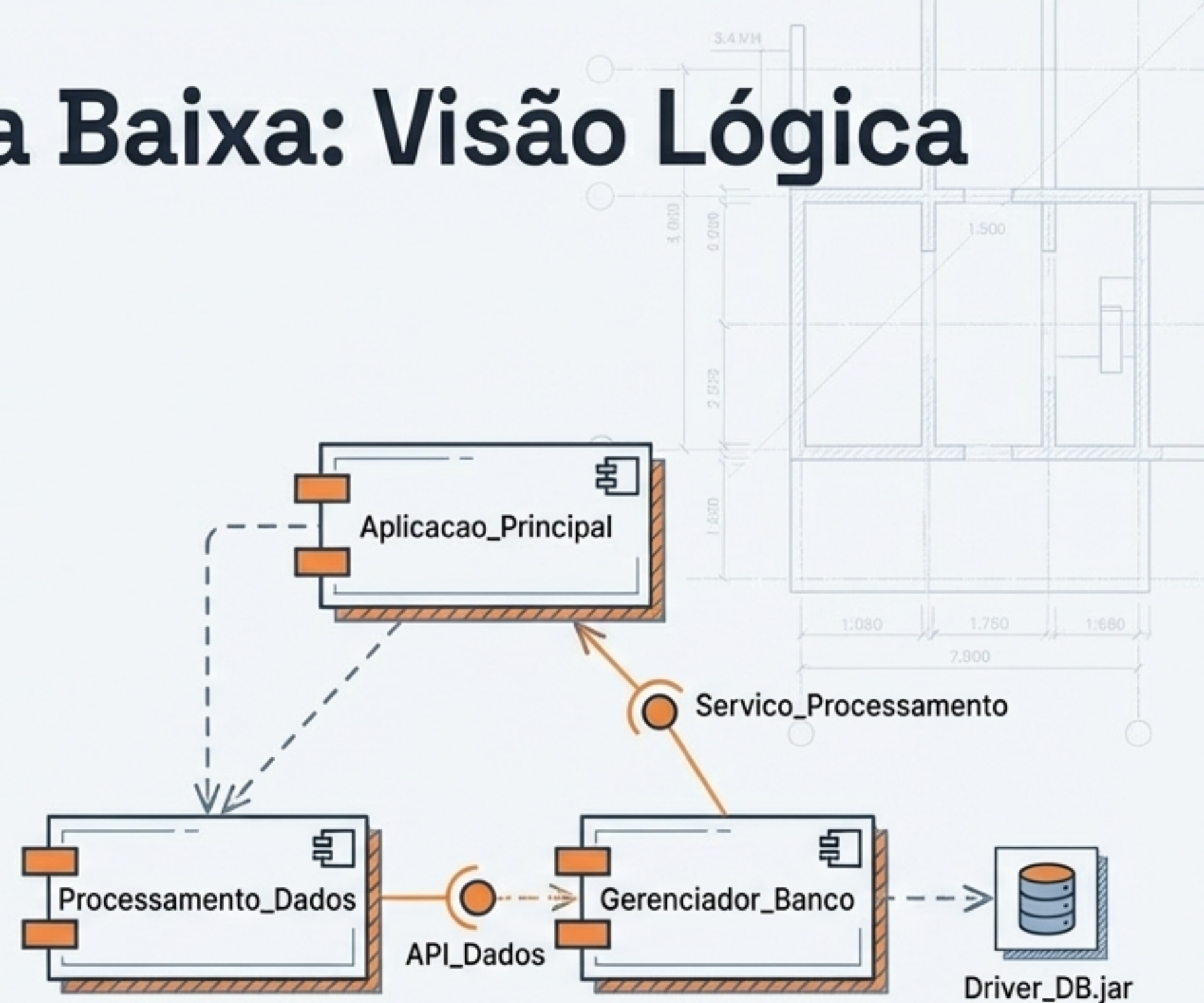


Conexões (A Cola)

Interfaces que unem todo o sistema.

Desenhando a Planta Baixa: Visão Lógica

- **Diagrama de Componentes (UML):** Foca na organização estrutural e modular do software, independente do hardware.
- **Componentes:** Os blocos de construção independentes (ex: pacotes, executáveis, bibliotecas .dll ou .jar) (JetBrains Mono for .dll and .jar).
- **Interfaces:** Os contratos ou APIs que os componentes expõem para conversar entre si.
- **A Regra de Ouro:** Visualizar a dependência entre módulos e entender como peças podem ser substituídas sem quebrar o todo.



Onde o Software Vai Morar: Visão Física

- **Diagrama de Implantação (UML)**: Modela a arquitetura física. Mostra onde o software será executado no mundo real.
- **Nós (Nodes)**: Os recursos de hardware e infraestrutura (Servidores, Computadores, Nuvem).
- **Artefatos**: O software compilado e real que é instalado dentro dos Nós.
- O Contraste Final:
 - **Componentes** = Composição do software (O Quê).
 - **Implantação** = Alocação na infraestrutura (Onde).



Grandes Padrões: Soluções Prontas para Problemas Conhecidos

O que são Padrões Arquiteturais? São templates ou soluções estruturais reutilizáveis e validadas pelo mercado para resolver problemas recorrentes.

O Padrão Clássico: Cliente-Servidor

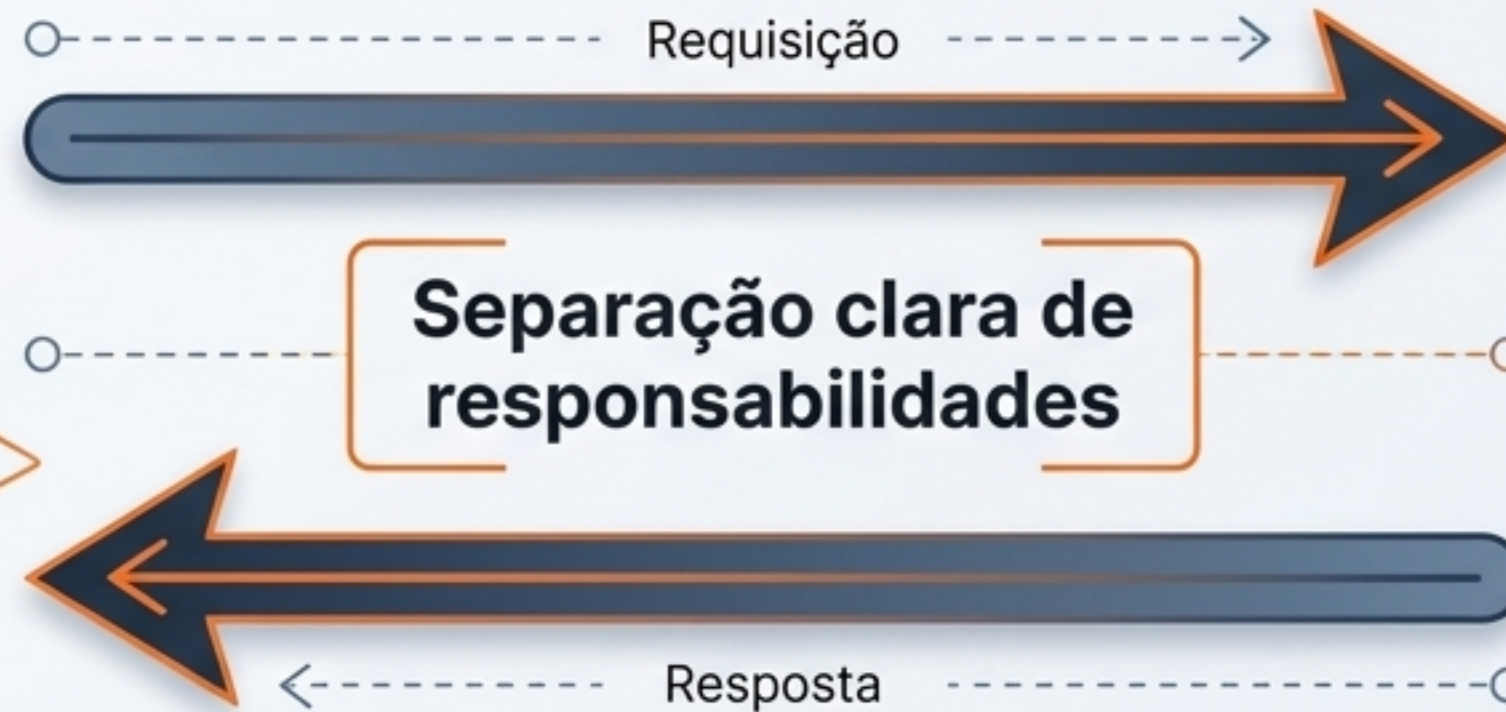
Cliente

Pede a informação
(Interface/App)



Servidor

Processa a lógica e
guarda os dados



O Padrão Monolítico: O Tudão

A Estrutura: Interface, lógica de negócios e acesso a dados compilados e implantados em um único pacote indivisível.



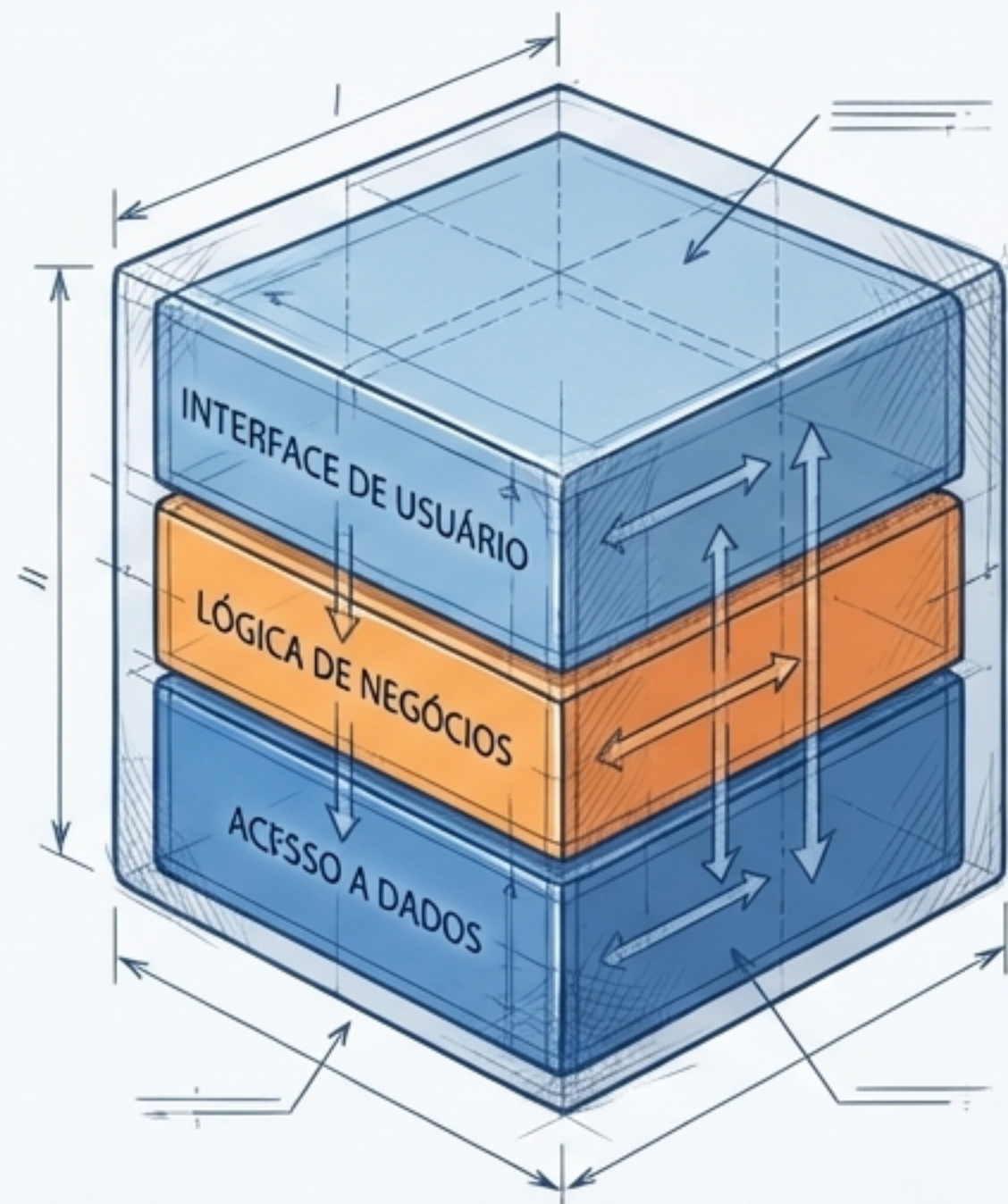
Vantagens:

- Simplicidade inicial (baixo custo).
- Fácil de desenvolver, testar e fazer o deploy.
- Excelente para começar um projeto (MVP).



O Gargalo:

- Quando o sistema cresce, o código vira um emaranhado.
- Qualquer mudança exige atualizar o sistema inteiro.
- Uma falha local pode derrubar a aplicação toda.



Microserviços: O Sistema Dividido

A Estrutura: O sistema é quebrado em vários serviços independentes, onde cada um executa uma tarefa específica.

✓ Vantagens:

- **Escalabilidade Cirúrgica:** Escale apenas a parte do sistema com alto tráfego.
- **Isolamento:** Se o serviço de pagamento cair, o catálogo continua no ar.

✗ O Custo (Trade-off):

- Altíssima complexidade de infraestrutura e operações (DevOps, redes).
- Não é recomendado para projetos pequenos.

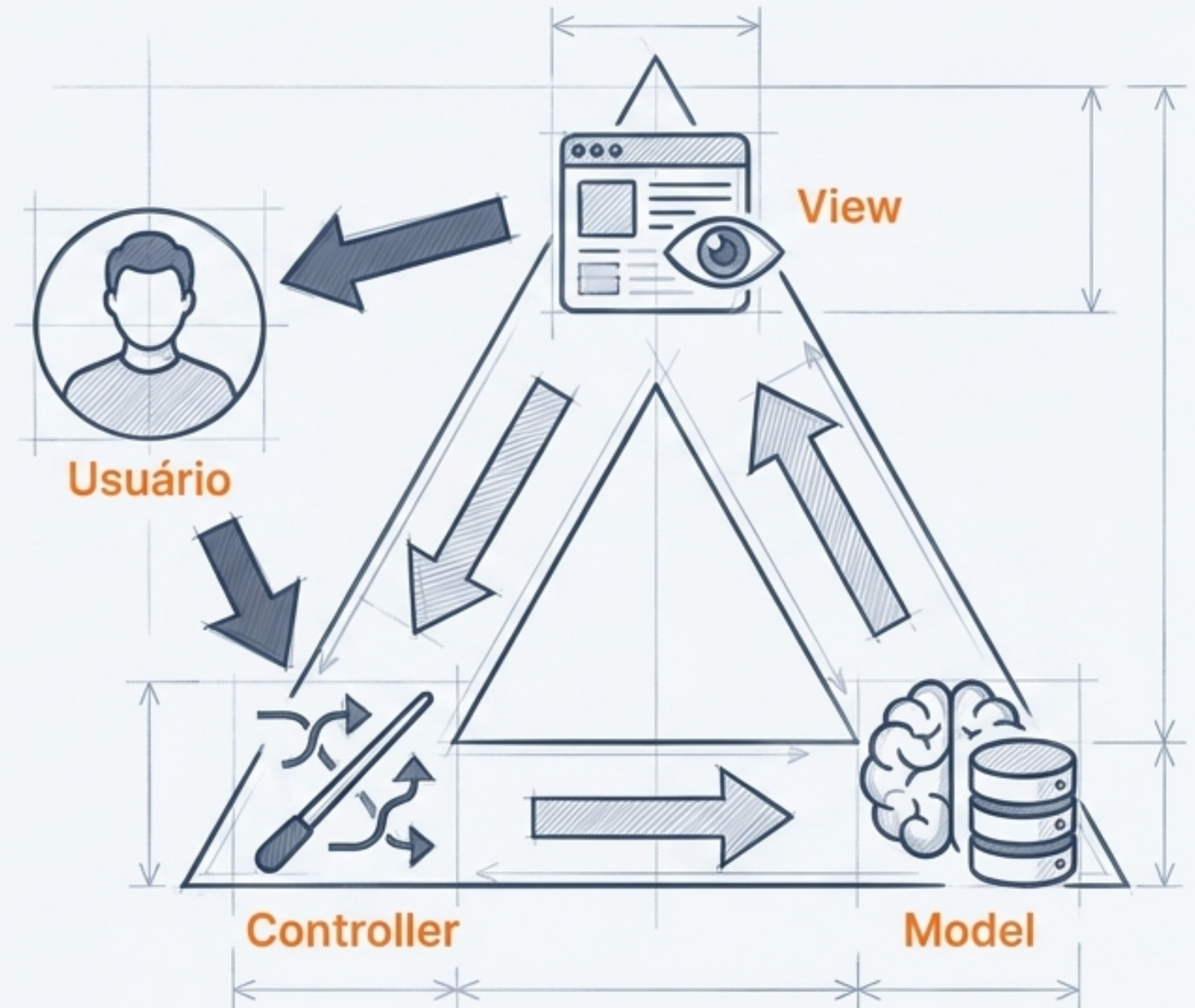


MVC: O Padrão Essencial da Web

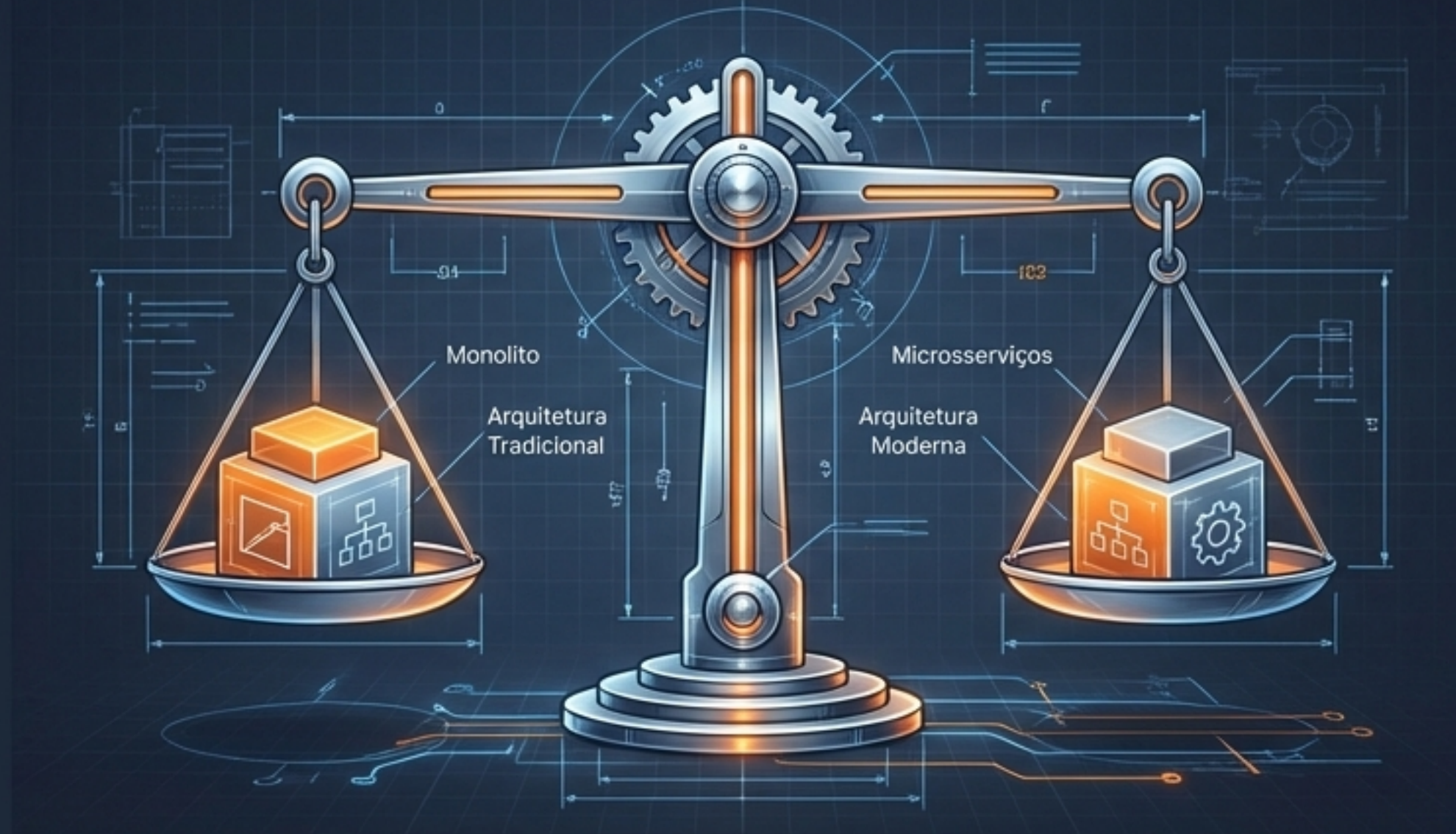
Model-View-Controller (MVC): Separação fundamental de preocupações estruturais.

- **Model (O Cérebro):** Gerencia os dados, o estado e as regras de negócio.
- **View (O Rosto):** A interface com o usuário (HTML/Telas). Recebe os dados e os exibe.
- **Controller (O Maestro):** O ponto de entrada. Recebe a requisição, pede dados ao Model e decide qual View exibir.

Vantagem: Permite que desenvolvedores Front-end e Back-end trabalhem em paralelo.



Conclusão: Não existe Bala de Prata



- **A arquitetura (o esqueleto)** que você escolhe hoje **define o sucesso, o custo e a sobrevivência** do seu software amanhã.
- **Não existe a melhor arquitetura universal.** Monolitos não são piores que Microsserviços; eles resolvem problemas diferentes.
- **O papel do Arquiteto:** Entender as perdas e ganhos e escolher o padrão certo para o modelo de negócio atual.